FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS UNIVERSIDAD DE CHILE

SPEL Space and Planetary Exploration Laboratory

SUCHAI 2, 3, PlantSat

Operation manual

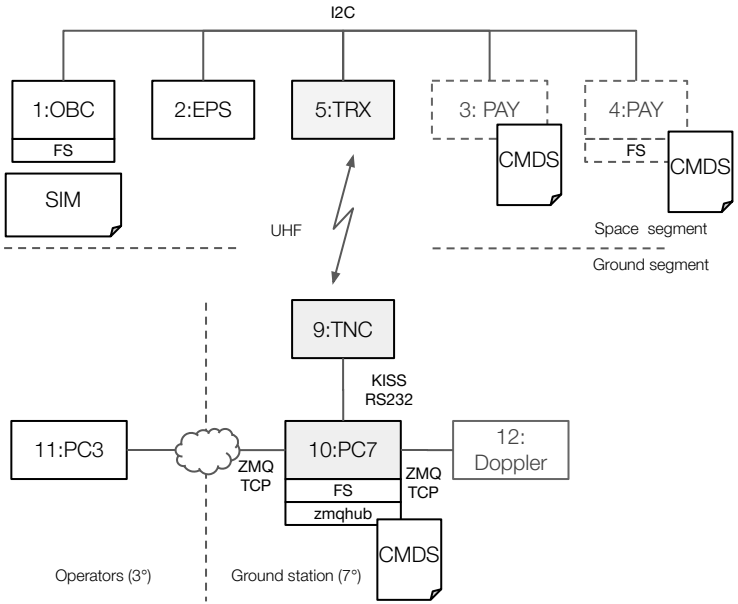
Facultad de Ciencias Físicas y Matemáticas
Universidad de Chile
Beaucheff 850, Santiago, Chile

www.spel.cl
@spel_uchile
spel@ing.uchile.cl

CAUTION: `com_send_cmd` COMMAND+PARAM max length: **248**

Commands and telecommands

Network diagram



Each node execute commands
Each node can send telecommands to other nodes

Execute a command in the ground station

- `tm_get_last <pay_id> # Print downloaded data`
- `tm_dump <pay_id> <file> # Download data as csv`

Send a command to other node

- `com_send_cmd <node> <command> [parameteres]`

`Example, from node 10 (GS)`

- `eps_get_hk`
- `com_send_cmd 1 eps_get_hk`

GS ask the EPS for HK data, EPS respond to GS

GS send cmd to OBC (1) to ask EPS for HK data. EPS respond to OBC

- `com_ping 10`
- `com_send_cmd 1 com_ping 10`

GS send ping to 10, GS respond to GS

GS send cmd to OBC (1), then OBC send ping to GS and GS respond back to OBC.

SPEL Space and Planetary Exploration Laboratory

CAUTION: FP resolution is **1 second** and **only one command** can be stored at a specific seconds. Periodical commands **may overwrite** other commands if timestamp collides.

PROTIP: Each node have a flight plan! You can set GS, OBC or Payload flight plan.

`fp_set_cmd_unix …` -> Set ground station FP
`com_send_cmd 1 fp_set_cmd_unix …` -> Set OBC FP

Flight plan

Unix time	<command> [parameters]
1651762597	com_ping 1
~~1651762600~~	~~com_ping 5~~
…	…
~~1651700000~~	~~com_ping 5~~
~~1651700010~~	~~com_ping 5~~
1651700020	com_ping 5
1651700040	other_command

`fp_set_cmd_unix 1651762597 1 0 com_ping 1`

`fp_set_cmd_unix 1651762600 1 0 com_ping 5`

`fp_del_cmd_unix 1651762600`

`fp_set_cmd_unix 1651700000 4 10 com_ping 5`

Overwrites!

CAUTION: `com_send_cmd` COMMAND+PARAM max length: **248**

INFO: `fp_set_cmd_unix` COMMAND+PARAM max length: **248+248**

INFO: `Max number of command in table`: **255**

INFO: `FP table is persistent`

Commands

Program **command** with **args** to specific **unixtime**, do **repeat** executions separeted **period** seconds

`fp_set_cmd_unix <unixtime> <repeat> <period> <command> [args]`

Delete **command** programmed at **unixtime** if exists

`fp_del_cmd_unix <unixtime>`

Delete **all** commands from flight plan

`fp_reset`

Send flight plan to **node** (node will print FP table to screen)

`tm_send_fp <node>`

Examples:

Download flight plan to ground station

`com_send_cmd 1 tm_send_fp 10`

Set flight plan to satellite OBC, so it ping GS 60 times each second

`com_send_cmd 1 fp_set_cmd_unix 1700000000 60 1 com_ping 10`

SPEL Space and Planetary Exploration Laboratory

Download telemetry data

PROTIP Execute a command in a remote node with
`com_send_cmd <node> <command> [args]`

Or using the shortcut:
`<node>: <command> [args]`

Payload ID de cada nodo de los satélites (para descargarlo)

```
    temp_sensors=0,        ///< 0: Temperature sensors
    ads_sensors,               ///< 1: Ads sensors
    eps_sensors,               ///< 2: Eps sensors
    status_sensors,          ///< 3: Status Variables
    stt_sensors,                ///< 4:
    rw_sensors,                ///< 5: RW Speed and current
sensor
    fss_sensors,                ///< 6: Ads sensors fss
    ekf_sensors,               ///< 7: Ads quat, omega
    ctrl_data,
    msg_sensors,             ///< 9: Store and forward msg
```

GS Payload ID de cada payload en la estación terrena (para verlo)
https://gitlab.com/spel-uchile/suchai-2-software/-/blob/framework/apps/groundstation/include/app/system/repoDataSchema.h#L595

```
tm_get_last <GS_PAY_ID>
tm_dump <GS_PAY_ID> <CSV_FILE>
```

6. From the **ground station** try the following commands to download data.

- Download first N samples from payload id to node 10

`com_send_cmd 1` **`tm_send_n`** `<payload_id> 10 <N>`

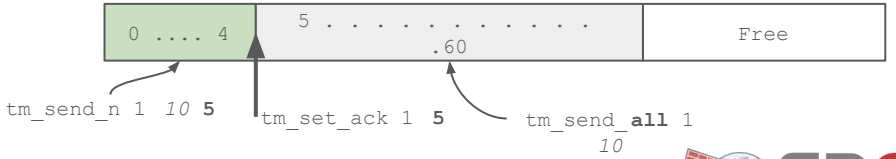
- Confirm 5 samples from payload

`com_send_cmd 1` **`tm_set_ack`** `<payload_id> 5`

- Download all samples from payload id to node 10

`com_send_cmd 1` **`tm_send_all`** `<payload_id> 10`

Sliding window schema to download and confirm data



Payloads status machine

Sensor id:

0: sen_get_temp
1: sen_get_adcs
2: sen_get_eps
3: sen_get_status
4: sen_get_adcs_fss

Activate a specific sensor

```
com_send_cmd 1 sen_activate <SENSOR_ID> <1|0>
```

Set status machine period and repetitions

```
com_send_cmd 1 sen_set_state 1 <PERIOD> <REPETITIONS|-1>
```

Example: take EPS data every 10 seconds, 1000 times

```
com_send_cmd 1 sen_activate 2 1
com_send_cmd 1 sen_set_state 1 10 10000
```

… Download later …

```
com_send_cmd 1 tm_send_all 2 10
```

… Dump table later …

```
com_send_cmd 1 tm_dump 22 eps_plantsat.csv
```

Transferring files

1. Files are fragmented using our custom FFP protocol and transferred as CSP packets.

2. Sending app calls the **tm_send_file** command to read, fragment and transmit the file. CSP packets are sent to port 9.

```
tm_send_file <path_to_file> <node>
```

This command uses the **com_send_file** function to fragment the file

3. Receiver app listens to file fragment packets in port 9 and calls **tm_parse_file** command to reconstruct the file according to the FFP protocol.

This is automatically done by the framework

4. If all fragments arrived correctly and in order, a new file is created in `<cwd>/recv_files/<file_id><filename>`

FFP File Fragmentation protocol

- nframe: Current frame number
- total: Expected frames
- type: START (100), DATA (101), END (102), PART(103)
- fileid: Random file identifier
- node: Node of origin

nframe 16 bit	type 8 bit	node 8 bit	fileid 16 bit	total 16 bit	data 192 bytes
0	100	1	32596	3	filename.txt\0\0...\0\0
1	101	1	32596	3	... data ...
2	101	1	32596	3	... data ...
3	102	1	32596	3	data … 0xAA0xAA0xAA0xAA

A. First frame (type 100) contains the file name (maximum 192 bytes including the null terminator).
B. Frames 1 to <total-1> (type 101) contain only binary data. All frames contain 192 data bytes.
C. Last frame <total> (type 102) contains up to 192 data bytes and is filled with trailing characters (0xAA) if necessary (must be removed to reconstruct the file)

SPEL Space and Planetary Exploration Laboratory

Transferring by parts

A.

nframe 16 bit	type 8 bit	node 8 bit	fileid 16 bit	total 16 bit	data 192 bytes

0	100	1	32596	3	filename.txt\0\0...\0\0
1	101	1	32596	3	... data ...
2	101	1	32596	3	... data ...
3	102	1	32596	3	data … 0xAA0xAA0xAA0xAA

SPEL Space and Planetary Exploration Laboratory

Commissioning

```
com_send_cmd 1
      drp_set_var_name dep_deployed 2      # Confirms deployment from GS
      drp_set_var_name obc_sw_wdt 0        # Reset software watchdog, send periodically
      tm_send_var 10 <var_name>
            dep_ant_deployed               # How many antennas were deployed, ans: [0, 4]
      tm_send_status 10                    # Send a status variable name to GS
      gssb_pwr 1 1                   # Turn on GSSB bus
      gssb_antenna_release <16|17|18|19> 2000 1 # Deploy any antenna (istage 1. On:2s, off:1s, rep:5)

com_set_time_node 1                        # Update node date time
tle_send 1 <SATNAME>                       # Download SATNAME TLE from celestrak and upload to node 1 (OJO)

com_send_cmd 1                             # Manually upload TLE
      tle_set <LINE 1>
      tle_set <LINE 2>
      tle_update
```

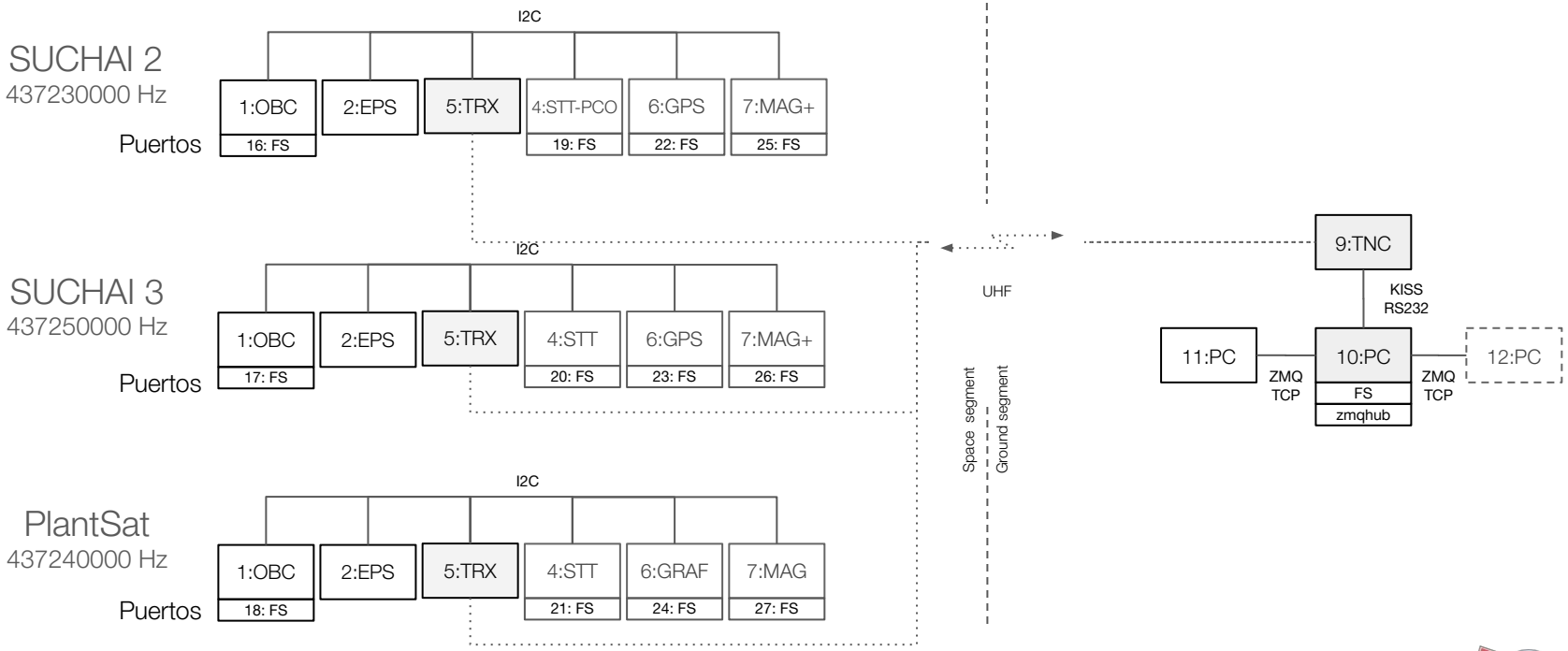
Data base

```
tm_dump <payload id> <csv_file>                 # Dump table to csv file

scp <from> <to>
      scp spel@groundstation:~/git/suchai-2-software/file.csv file_<date>.csv
      scp spel@groundstation:~/git/suchai-2-software/suchai.10.db suchai.10.<date>.db

sqlite browser -> Permite explorar la base de datos sqlite
```

Payloads operations

CSP Address size: 5 bits => 32 nodes
CSP Port size 6 bits => 64 ports

SUCHAI 2, 3, PlantSat network diagram